/// Plan de implementación del frontend Angular dividido en lotes ejecutables de forma incremental, cada uno dejando la app funcional e interactuando con el backend real antes de pasar al siguiente.

Plan de lotes — Frontend Angular (AlojaME)

Decisiones de base (confirmadas al inicio)

- **Rama de trabajo:** `FinalBranch` de este repositorio.
- **Ubicación del proyecto Angular:** carpeta `frontend/` en la raíz del repo, hermana de `GA/` (el backend). El repo queda: `GA/`, `frontend/`, `Archives/`.
- **Angular 22**, componentes standalone, sin librería de UI (sin Angular Material, sin Bootstrap/Tailwind). CSS puro.
- **Estética: después.** Primero vistas simples y funcionales, preparadas para darles identidad visual más adelante (Lote 6).
- **Sin tests automatizados por ahora** (mismo criterio que el backend). Se puede sumar cobertura después si el TFI lo pide.
- **Backend real:** corre en `http://localhost:8081` (`server.port=${SERVER_PORT:8081}`). CORS (`cors.allowed-origins`) ya tiene como default `http://localhost:4200` (puerto estándar del dev server de Angular) — no hace falta tocar el backend para que el front pegue contra la API real.
- **El frontend se construye desde cero** (un intento previo se descartó por decisión del equipo).

Por qué estos lotes y en este orden

Cada lote corresponde 1:1 a una Historia del tablero Jira (proyecto SCRUM) y, dentro de cada uno, las subtareas van en orden de dependencia (lo que las demás vistas necesitan, primero). Cada lote cierra con la app funcionando de punta a punta para lo que cubre — no queda nada a medio hacer de un lote a otro. El Lote 1 es el único que no se corresponde con un rol: es la base que hace falta antes de que cualquier vista funcione (routing, guards, interceptor, layout) más las vistas públicas, así que al cerrarlo ya hay una app real con la que interactuar (navegar, buscar, registrarse, verificar cuenta, loguearse) antes de construir nada específico de un rol.

Lote 1 — Base ejecutable (infraestructura + público + autenticación) — ✅ TERMINADO

Jira: Historia SCRUM-13 (Infraestructura base) + Historia SCRUM-14 (Vistas públicas y autenticación).

Orden de implementación:

1. **SCRUM-18 — Setup inicial del proyecto Angular 22.** Routing base, estructura de carpetas por feature, cliente HTTP configurado contra `http://localhost:8081`.
2. **SCRUM-19 — AuthGuard + RoleGuard.** Guards de ruta: sesión activa y rol correspondiente, redirigiendo a `/login` o `/403`.
3. **SCRUM-20 — Interceptor HTTP (JWT + manejo de 401).** Adjunta el JWT a cada request, redirige a `/login` si la API responde 401.
4. **SCRUM-21 — Layout/navbar por rol + páginas de error 403/404.** Menú que cambia según el rol autenticado, logout, páginas de error.
5. **SCRUM-22 — Home.** RF14. Listado inicial de alojamientos disponibles.
6. **SCRUM-23 — Búsqueda con filtros.** RF14, RF39. Filtro por rango de precio, capacidad y servicios.
7. **SCRUM-24 — Detalle de alojamiento.** RF15, RF16, RF17, RF35. Galería, servicios, reseñas, botón de reservar (sin sesión redirige a login).
8. **SCRUM-25 — Registro de cuenta.** RF01. Formulario de registro como cliente o anfitrión.
9. **SCRUM-26 — Verificación de cuenta.** RF02. Confirmación de token recibido por email, opción de reenvío.
10. **SCRUM-27 — Inicio de sesión.** RF03. Login con email y contraseña, guarda el JWT.

Criterio de cierre del lote: con el backend corriendo, se puede levantar el front, navegar sin sesión, buscar y ver el detalle de un alojamiento real de la base de datos, registrarse, verificar la cuenta por el email real (Brevo) y loguearse, quedando autenticado con JWT real.

Verificado end-to-end con Playwright contra el backend real (no solo `ng build`): registro → token real leído de la tabla `token_verificacion` → verificación → login → sesión persistida en `localStorage` y recuperada tras recargar la página → logout; búsqueda con filtro de precio achicando resultados reales; detalle de alojamiento con galería/servicios/reseñas reales; botón "Reservar" redirige a `/login` con `returnUrl` si no hay sesión, y a la ruta protegida por `roleGuard(['CLIENTE','ADMIN'])` si la hay; página 404.

Cambio de alcance respecto del plan original: `/Alojamiento/listar` y el resto de los GET de catálogo/galería/reseñas **no eran públicos** en el `SecurityConfig` heredado del backend (exigían rol autenticado) — lo cual hacía imposible la navegación "sin sesión" que este lote asume. Decisión explícita del equipo: abrirlos a público (modelo Airbnb), documentada en el error #13 del backend en *Bitacora_Tecnica_AlojaME.pdf* y en el commit correspondiente. Es el único cambio de backend de este lote; todo lo demás es frontend puro.

Bug real encontrado y resuelto durante la verificación: `NG0200` (dependencia circular) en `AuthService` al recargar la página con sesión activa, que borraba la sesión en cada F5 sin ningún error visible en la UI. Ver error #1 del frontend en *Bitacora_Tecnica_AlojaME.pdf* para el diagnóstico completo y el fix.

Lote 2 — Vistas del rol Cliente — ✅ TERMINADO

Jira: Historia SCRUM-15.

Orden de implementación:

1. **SCRUM-28 — Mi perfil.** RF04, RF05. Ver y editar datos personales. ✅ Finalizado en Jira, verificado end-to-end con Playwright (editar telefono, persiste tras reload).
2. **SCRUM-29 — Formulario de reserva.** RF18, RF19. Selección de fechas, cálculo de precio en vivo, validación de solapamiento contra la API real. ✅ Finalizado en Jira, verificado (reserva creada contra un alojamiento real; segunda reserva con las mismas fechas rechazada por la API con el mensaje real de solapamiento).
3. **SCRUM-30 — Mis reservas (listado).** RF26. ✅ Finalizado en Jira, verificado.
4. **SCRUM-31 — Detalle de reserva (cliente) + comprobante.** RF21, RF22, RF29, RF31, RF40. Estado, pago, cancelar, vista imprimible del comprobante. SCRUM-33 (dejar reseña) se resolvió inline en este mismo componente, no como pantalla separada. ✅ Finalizado en Jira, verificado de punta a punta: cancelar una reserva SOLICITADA (gotcha #12), y la sección de pago mostrando el desglose real (monto base, comisiones, total, estado PAGADO con fecha) contra una reserva confirmada y pagada de verdad con MercadoPago.
5. **SCRUM-32 — Checkout / resultado del pago.** RF29. Pantalla `/pago-resultado` (ruta y query param `estado` fijados por el propio backend en `MercadoPagoService.crearPreferencia`) ✅ Finalizado en Jira, verificado en los 3 casos de query param Y en el ida-y-vuelta real: preferencia real creada (`urlCheckout` real de MercadoPago), pago completado a mano por el usuario en el checkout real (sandbox, tarjeta de prueba) con resultado aprobado.
6. **SCRUM-33 — Dejar reseña.** RF34. Resuelto inline dentro de SCRUM-31 (ver arriba). ✅ Finalizado en Jira, verificado: reseña enviada contra una reserva real, persiste, el formulario desaparece y muestra "Ya dejaste una reseña" tras recargar.
7. **SCRUM-34 — Resumen de mis gastos.** RF38. ✅ Finalizado en Jira, verificado.

Verificación de pago real, de punta a punta (con el usuario completando el pago a mano): reserva creada por un cliente real → confirmada por un anfitrión real (dispara `crearPagoParaReserva`, llamada real a `POST /checkout/preferences` de MercadoPago) → el usuario pagó en el Checkout Pro real (sandbox) con una tarjeta de prueba → webhook simulado a mano contra `POST /Pago/webhook` con el id de operación real (`procesarNotificacionPago` vuelve a consultar el estado contra la API real, no confía en el body) → `Pago` quedó `PAGADO` con fecha real → verificado en la UI del detalle de reserva. Ver error #2 del frontend en *Bitacora_Tecnica_AlojaME.pdf* para la metodología completa (bloqueos de red del entorno, anti-bot del checkout, simulación del webhook).

Limitación encontrada, fuera de alcance de este lote: la API de Payouts de MercadoPago (`crearPayout`, usada por `finalizar()` para pagarle al anfitrión) devuelve `400 Bad Request: "Invalid signature"` con las credenciales TEST de esta app — limitación de la cuenta de MercadoPago (necesita habilitación extra), no un bug de código. Confirmado que `crearPreferencia` / `consultarPago` /el webhook sí funcionan perfecto con las mismas credenciales. Documentado como error #14 del backend en *Bitacora_Tecnica_AlojaME.pdf*. Para probar `finalizar()` /reseña en este lote, se forzó el estado `FINALIZADA` directo en la base de datos de prueba (sin pasar por el Payout roto) — es un parche de testing, no afecta el código de producción. Este mismo límite va a reaparecer en el Lote 3 cuando se implemente la vista de "confirmar/finalizar reserva" del anfitrión.

Criterio de cierre del lote: un cliente logueado puede reservar un alojamiento real, confirmar y pagar con MercadoPago real (modo sandbox/test), ver el estado de su reserva,

dejar una reseña una vez finalizada, y consultar su resumen de gastos. **Cumplido y verificado de punta a punta.**

Lote 3 — Vistas del rol Anfitrión — ✅ TERMINADO

Jira: Historia SCRUM-16.

Orden de implementación:

1. **SCRUM-35 — Mis alojamientos (listado).** RF11, RF13. ✅ Finalizado en Jira, verificado (listado + activar/desactivar).
2. **SCRUM-36 — Alta / edición de alojamiento.** RF11, RF12, RF17. Datos, dirección, tipo, servicios asociados. ✅ Finalizado en Jira, verificado (alta real, edición con precarga, agregar/quitar servicios). La dirección no es editable despues de creada (el backend no tiene endpoint para eso, `AlojamientoModificarDTO` no trae ese campo) — decisión de alcance, no limitación encontrada.
3. **SCRUM-37 — Gestión de galería de imágenes.** RF16. Carga vía Cloudinary real. ✅ Finalizado en Jira, verificado de punta a punta con credenciales reales de Cloudinary del usuario: subida real (URL real de `res.cloudinary.com`, imagen accesible públicamente), renderizado en la UI, y borrado confirmado tanto en la UI como en la base.
4. **SCRUM-38 — Reservas de mis alojamientos (listado).** RF27. Filtro por estado. ✅ Finalizado en Jira, verificado contra datos reales.
5. **SCRUM-39 — Detalle de reserva (anfitrión) — confirmar/cancelar/finalizar.** RF20, RF23, RF25. Finalizar dispara el payout real. ✅ Finalizado en Jira. Confirmar y cancelar verificados de punta a punta contra MercadoPago real. Finalizar fue probado real y falla con gracia (mensaje claro, sin corromper el estado de la reserva) por la limitación de la API de Payouts de MercadoPago documentada en el error #14 del backend en *Bitacora_Tecnica_AlojaME.pdf* — no es un bug de este código, es una limitación de la cuenta de MercadoPago usada para las pruebas (necesita habilitación extra tipo marketplace/collector).
6. **SCRUM-40 — Cobros pendientes.** RF33. ✅ Finalizado en Jira, verificado contra datos reales (reservas pagadas sin liquidar, consistente con la limitación de Payouts del punto anterior).
7. **SCRUM-41 — Resumen de mis alojamientos.** RF37. ✅ Finalizado en Jira, verificado (ingresos, cantidad de reservas y rating promedio reales, incluída una reseña real dejada en el Lote 2).

Criterio de cierre del lote: un anfitrión logueado publica un alojamiento real con imágenes, confirma una reserva (dispara la preferencia de pago real), la finaliza (dispara el payout real) y consulta sus cobros y resumen. **Cumplido, con la única salvedad de que el payout real al finalizar no se puede completar en este entorno por la limitación de la cuenta de MercadoPago (ver gotcha #14) — la transición de estado y toda la UI funcionan correctamente, lo que falla es específicamente la llamada externa a `/v1/payouts`.**

Lote 4 — Vistas del rol Administrador — ✅ TERMINADO

Jira: Historia SCRUM-17.

Orden de implementación:

1. **SCRUM-42 — Gestión de usuarios.** RF06, RF07. ✅ Finalizado en Jira, verificado: listado con filtro por tipo, activar/desactivar, editar cualquier usuario, crear otro administrador (login verificado con el admin nuevo).
2. **SCRUM-43 — Catálogo de tipos de alojamiento.** RF09. ✅ Finalizado en Jira. **Hallazgo:** el backend solo tenía `listar`/`reemplazar`/`eliminar` (sin alta ni edición), pese a que RF09 pide explícitamente las 3 — se agregaron `POST /crear` y `PUT /{id}` al backend (ver error #15 del backend en *Bitacora_Tecnica_AlojaME.pdf*). Verificado: alta, edición en el lugar, fusionar duplicados, eliminar, y rechazo claro al duplicar un nombre.
3. **SCRUM-44 — Catálogo de servicios.** RF10. ✅ Finalizado en Jira, mismo criterio y mismo hallazgo que SCRUM-43 (gotcha #15). Verificado completo, incluido el rechazo de borrar un servicio en uso ("Utilice reemplazar").
4. **SCRUM-45 — Todas las reservas (admin).** RF24. ✅ Finalizado en Jira, verificado (listado global con filtro por estado, de cualquier cliente/anfitrión).
5. **SCRUM-46 — Reembolso manual.** RF24. ✅ Finalizado en Jira. El llamado llega correctamente formado hasta la API real de reembolsos de MercadoPago (confirmado vía el `404 resource not found` que devuelve, por tratarse de un `id_pago_externo` sintético de una reserva marcada `PAGADA` a mano en el Lote 3 para probar `finalizar()` — no un pago real); decisión explícita del equipo de no repetir el ciclo de pago real para esta subtarea. La UI maneja el error con gracia.
6. **SCRUM-47 — Flujo de dinero.** RF32. ✅ Finalizado en Jira, verificado: facturación, comisiones, liquidado/reembolsado, ingresos mensuales por mes y cancelaciones por motivo, todo contra datos reales acumulados en los Lotes 2-4.
7. **SCRUM-48 — Estadísticas globales.** RF36. ✅ Finalizado en Jira, verificado: rankings de alojamientos por ingresos, cantidad de reservas y rating promedio.

Criterio de cierre del lote: el administrador tiene control total: gestiona usuarios y catálogos, cancela cualquier reserva, define reembolsos manuales y consulta las estadísticas/flujo de dinero reales de la plataforma. **Cumplido.**

✅ Los 4 lotes funcionales están TERMINADOS

Los Lotes 1 a 4 (infraestructura, Cliente, Anfitrión, Administrador — 41 subtareas de Jira en total entre SCRUM-18 y SCRUM-48) están implementados y verificados de punta a punta contra el backend real, MercadoPago real (sandbox) y Cloudinary real. Lo único pendiente del frontend funcional es lo que ya estaba confirmado como "se hace después":

Lote 4.1 — Auditoría completa, conexión back↔front y README — ✅ TERMINADO

Pedido explícito antes de encarar el Lote 5: auditar backend y frontend completos, verificar la conexión entre ambos, crear el README, y recorrer en vivo cada camino de cada rol.

1. **Auditoría backend.** Regla Service→Repository re-verificada sin violaciones en los 12 Services. Se armó el inventario completo de endpoints (los 11 `RestController`) y se cruzó

uno por uno contra `SecurityConfig`: todos tienen un matcher explícito y deliberado, ninguno cae sin querer al `anyRequest().authenticated()` genérico. Único hallazgo (cosmético, no explotable): `POST /Alojamiento/{id}/servicios` admite el rol `CLIENTE` a nivel HTTP, pero `verificarDuenioOAdmin` en el controller ya bloquea a cualquiera que no sea dueño o admin — queda documentado, no se tocó. `mvn clean compile` limpio, sin `TODO`/`FIXME`/prints de debug reales (los únicos matches eran la palabra "TODOS" en comentarios).
2. **Auditoría frontend.** Navbar y `roleGuard` de cada ruta verificados consistentes rol por rol. `ng build` limpio (con Node 24, ver nota de infraestructura abajo), sin warnings. Sin código de debug ni placeholders reales.
3. **Conexión back↔frontend.** Se listaron todas las llamadas HTTP de cada servicio Angular y se cruzaron contra los endpoints reales del backend. Todo matchea, con dos casos explicados (no son bugs): `DELETE /Alojamiento/eliminar/{id}` no se usa desde el frontend porque "Mis alojamientos" usa un toggle activo/inactivo (soft delete) en su lugar; `GET /Pago/reserva/{id}` tampoco se usa porque el detalle de reserva ya trae los datos de pago embebidos en el DTO de Reserva.
4. **Hallazgo real (no cosmético):** `POST /Usuario/alias-cobro/validar` y `/confirmar` — el flujo completo de 2 pasos para que un anfitrión registre su alias/CBU/CVU de cobro — **no tenía ninguna pantalla en el frontend**. Sin esto, un anfitrión no tiene forma real de indicarle a la plataforma a dónde pagarle. Se decidió resolverlo en este mismo lote (no esperar al Lote 5): se agregó la sección "Alias de cobro" en `mi-perfil` (visible solo para `ANFITRION`/`ADMIN`), con el mismo flujo de 2 pasos del backend. Verificado en vivo contra el backend real con un CBU de prueba: identificó el banco (BBVA Argentina) y persistió el alias tras confirmar. Ver error #3 del frontend en *Bitacora_Tecnica_AlojaME.pdf*.
5. **Recorridos completos por rol, en vivo contra el backend real:**
   - **Visitante anónimo:** home → buscar (6 resultados) → detalle de alojamiento → intento de reservar sin sesión (redirige a `/login` con `returnUrl` preservado, tanto por click como por URL directa). Sin errores de consola.
   - **Cliente:** registro real → verificación con token leído de la tabla `token_verificacion` → login → perfil (sección de alias de cobro correctamente oculta, es rol CLIENTE) → búsqueda → detalle → formulario de reserva (rechazó fechas superpuestas con el mensaje correcto, aceptó fechas libres) → detalle de la reserva creada (estado SOLICITADA, comprobante, cancelar) → listado de "Mis reservas" → "Mis gastos" (en 0, correcto porque la reserva nueva todavía no tiene pago).
   - **Anfitrión:** login → "Mis alojamientos" → galería → "Reservas recibidas" → confirmó en vivo la reserva que acababa de crear el Cliente del punto anterior (SOLICITADA → CONFIRMADA, se creó el Pago PENDIENTE con el monto base correcto) → "Cobros pendientes" (total correcto) → "Resumen" (ingresos/cantidad/rating correctos) → alias de cobro (punto 4).
   - **Administrador:** login → Usuarios, Catálogo de tipos, Catálogo de servicios, Todas las reservas (+ detalle de una reserva ajena), Flujo de dinero, Estadísticas globales (ya verificada contra datos reales en el Lote 4). Todas las pantallas renderizan sin errores.
6. **README.md creado** en la raíz del repo: stack, estructura, cómo correr backend y frontend, qué puede hacer cada rol, resumen del modelo de dominio, estado del proyecto y pendientes conocidos.
7. **Nota de infraestructura (no es del código):** el Angular CLI 22 exige Node 22.22.3+/24.15+/26+; el Node del `PATH` por default en este entorno es 22.22.0 y hace

fallar `ng build` con un mensaje de versión mínima. Hay un Node 24.22.2 instalado en `/opt/nvm/versions/node/v24.22.2/bin` — hay que anteponerlo al `PATH` para compilar (`export PATH=/opt/nvm/versions/node/v24.22.2/bin:$PATH`). `ng serve` ya corría con ese Node desde que se levantó por primera vez en este entorno, por eso el síntoma solo aparecía al correr `ng build` manualmente.

Criterio de cierre del lote: backend y frontend auditados sin hallazgos de seguridad reales, conexión back↔front verificada llamada por llamada, el único gap funcional real (alias de cobro) resuelto y verificado en vivo, README creado, y los 4 roles recorridos de punta a punta contra el backend real sin errores. **Cumplido.**

Lote 5 — Responsive — ✅ TERMINADO

Sprint "Frontend responsive - Lote 5" en Jira (epic SCRUM-49, historias SCRUM-50 a SCRUM-58). Cada historia es un commit propio con la clave en el mensaje.

Historia	Qué se hizo
SCRUM-50	Base mobile-first en `styles.css`: breakpoints 768px (tablet) y 1024px (desktop), controles de 44px de alto, inputs de 16px (sin zoom automático en celulares), tipografía fluida, imágenes que no desbordan.
SCRUM-51	Navbar: menú desplegable debajo de 1024px (se cierra al navegar y con Escape); en desktop los links pasan a otra línea en vez de desbordar (el menú de admin desbordaba 572px en tablet).
SCRUM-52	Inicio y búsqueda: filtros plegables en mobile con contador de filtros activos; barra lateral fija en desktop.
SCRUM-53	Detalle de alojamiento: galería en carrusel deslizable en mobile y mosaico desde tablet; barra de reserva fija al pie en mobile y columna lateral en desktop. Se corrigió el `returnUrl` con doble barra al reservar sin sesión y el login ahora solo vuelve a rutas internas.
SCRUM-54	Formularios: una columna en mobile, grilla de 2-3 columnas desde tablet (`.form-grid`), autocompletado del navegador, teclados numéricos, etiquetas en los campos de dirección.
SCRUM-55	Tablas de listados como tarjetas debajo de 1024px (`.table-cards` + `data-label`), de a dos por fila en tablet; estilo de tabla único en `styles.css`.
SCRUM-56	Paneles: tarjetas de métricas (`.kpi-grid`), ingresos mensuales como barras (antes, tabla de 12 columnas), rankings y totales que se reacomodan.
SCRUM-57	Galería del anfitrión en miniaturas, detalles de reserva como lista etiqueta/valor, acciones apiladas a todo el ancho en mobile.
SCRUM-58	Verificación completa y ajustes finales (filtros compactos en desktop, márgenes de páginas de error y del contenido).

Verificación (SCRUM-58). Con Playwright contra el backend real: las 33 rutas de la app con los 4 perfiles (visitante, cliente, anfitrión, administrador) en 4 anchos (360, 768, 1024 y 1280px), 132 casos en total. En cada caso se midió scroll horizontal de la página, tablas que necesitan scroll lateral y botones/inputs/selects de menos de 44px de alto. Resultado: **132/132 sin hallazgos** (antes de empezar: navbar de admin desbordando en tablet y desktop, 6 tablas

con scroll lateral en mobile y controles chicos en casi todas las pantallas). Además se repitieron en 360px los recorridos clave (login, menú, listado → detalle, edición en línea de catálogo, cambio de año en flujo de dinero) sin errores de consola.

Lo que no entra en este lote: la paleta sigue siendo neutra (escala de grises); la identidad visual llega en el Lote 6. La estructura responsive no depende de los colores, así que se puede cambiar la paleta tocando solo las variables de `:root` en `styles.css`.

Lote 6 — Identidad visual, fotos y mapa — ✅ TERMINADO

Sprint "Identidad visual - Lote 6" en Jira (epic SCRUM-59, historias SCRUM-60 a SCRUM-69). Cada historia es un commit propio con la clave en el mensaje. El diseño se probó antes en un laboratorio aparte (una copia en HTML, fuera del proyecto) y recién después se pasó a la app.

Kaza es la mascota de AlojaME: una carita adentro de una casa con techo. Es la marca de la app y va siempre de un solo color, amarillo (`#FDCA40`) sobre fondos oscuros y negro (`#080708`) sobre fondos claros. Tiene tres versiones en `shared/kaza`: "trazo" (logo del encabezado y del pie, estados vacíos), "maciza" (tamaños chicos: menú, "Cargando...", favicon) y "avatar" (la única excepción de color: cara amarilla con aro negro, porque queda mitad sobre el encabezado negro y mitad sobre el fondo claro).

Historia	Qué se hizo
SCRUM-60	Backend: `imagenPrincipal` en la respuesta de alojamientos (primera foto de la galería), resuelta con una sola consulta agrupada para todo el listado. `AlojamientoService` la pide a `GaleriaService`, nunca al repositorio de imágenes.
SCRUM-61	Backend: `latitud` y `longitud` opcionales en la dirección, validadas en rango y siempre juntas. Se cargan al publicar y se pueden corregir al editar.
SCRUM-62	Base visual: paleta (negro, tomate, acero, dorado y azul, con variantes que cumplen contraste AA), Plus Jakarta Sans, botones (`.btn--primario`, `.btn--peligro`), campos, tablas, métricas, etiquetas de estado con color (`app-estado`), favicon con Kaza y formato argentino (`es-AR`: $45.500, 14/12/2026).
SCRUM-63	Encabezado negro fijo: logo con Kaza, buscador siempre a mano (`/buscar?q=...`) y avatar de Kaza que sobresale; el menú de la cuenta agrupa las opciones por rol. Reemplaza a la barra de links.
SCRUM-64	Pie de página (EloySoft, una división de RoldaCorporation, Mar del Plata), "Sobre nosotros" y "Ayuda y legales" (preguntas frecuentes, cancelaciones, pagos, arrepentimiento, términos, privacidad y accesibilidad). Los textos describen lo que la app hace de verdad.
SCRUM-65	Tarjetas con foto (miniatura de Cloudinary, carga diferida, Kaza si no hay fotos), tipo, ciudad, huéspedes y precio. Inicio con portada. "Mis alojamientos" usa la misma tarjeta.
SCRUM-66	Búsqueda con modo "Lista / Mapa": Leaflet + OpenStreetMap, precios sobre el mapa (puntos cuando el mapa está alejado), ficha con foto y link al detalle. El mapa se descarga solo al elegirlo (`@defer`).
SCRUM-67	Formulario de alojamiento: mapa para marcar o mover el punto y botón "Buscar en el mapa" (Nominatim, un pedido por vez, nunca mientras se escribe; el pedido no lleva el token de sesión).

Historia	Qué se hizo
SCRUM-68	Esqueletos de carga en todas las pantallas, estados vacíos con Kaza y avisos breves al iniciar/cerrar sesión, guardar, cancelar, confirmar, etc. Animaciones cortas que se apagan con "reducir movimiento".
SCRUM-69	Tildes en todos los textos visibles, título propio por pantalla (`TitleStrategy`), coordenadas en `datos_de_prueba.sql` y `datos_demo_mar_del_plata.sql`, verificación final.

Peso de la app. El bundle inicial quedó en ~470 kB (antes del lote, 473 kB): Leaflet, "Sobre nosotros", "Ayuda y legales" y las pantallas de anfitrión y de administración se descargan recién cuando se usan.

Verificación (SCRUM-69). Con Playwright contra el backend real y la demo de Mar del Plata cargada: las 33 rutas con los 4 perfiles en 360, 768, 1024 y 1280px, 132 casos. En cada caso se midió scroll horizontal, controles de menos de 44px y errores de consola: **132/132 sin hallazgos**. Además se probaron de punta a punta: buscador del encabezado, menú de la cuenta (cierre con Escape y al tocar afuera), links del pie a cada sección de Ayuda, modo mapa con filtros (72 alojamientos de la demo), alta de un alojamiento con "Buscar en el mapa" y edición moviendo el punto (verificado lo guardado en la API), esqueletos con carga lenta simulada, estados vacíos y avisos de inicio y cierre de sesión. En el build de producción se confirmó que el mapa se descarga recién al elegir "Mapa".

Ajustes finales, después del Lote 6 — ✅ TERMINADO

Sin sprint propio en Jira: son mejoras y correcciones surgidas al probar la app completa antes de la demostración. El detalle de cada decisión está en *Documentacion_Frontend_AlojaME.pdf*.

Tema	Qué se hizo
Accesos y roles	Auditoría de cada recorrido por rol. El administrador ya no puede reservar (el backend lo rechazaba). `soloVisitanteGuard` e `idNumericoGuard`. La sesión se cierra sola al vencer el token o si la cuenta se desactiva.
Reservas	Calendario de rango propio (días ocupados y pasados en gris, días intermedios marcados), también para elegir períodos en el flujo de dinero. Cambio de fechas desde el detalle de la reserva. Fechas siempre en hora local.
Carta de avisos	Todas las confirmaciones y resultados en una misma carta, con Kaza amarilla como insignia (entrada en dos tiempos y balanceo). Antes de mover dinero explica qué va a pasar, después qué pasó.
Pagos simulados	Con `PAGOS_SIMULADOS=true` (único cambio de backend de esta etapa), reembolsos y liquidaciones se registran sin MercadoPago, que en modo de prueba no los permite. La carta lo aclara.
Baja en cascada	Desactivar un usuario pausa sus alojamientos y cancela sus reservas sin pago; las pagadas quedan para que el administrador decida.
Administración	Ficha de usuario (números, alojamientos a su nombre y reservas). Alojamiento, cliente y anfitrión son links en todas las listas de reservas. Catálogos y gestión de usuarios más compactos.
Búsqueda	Paginado de a 12 resultados y servicios elegidos en una carta con buscador.

Tema	Qué se hizo
Gráficos	Chart.js con una frase de conclusión arriba, el valor escrito en cada barra, la más alta resaltada y un botón "i" con la explicación de cada gráfico.
Datos de demostración	`datos_demo_completa.sql`: todos los casos de cada rol, con fotos de casas, y una guía de cuentas al terminar.

Pendientes conocidos, no bloqueantes

- **Trabajo individual**, autorizado explícitamente por la cátedra — el punto de la consigna sobre "historial real de commits de cada integrante" no aplica, es un solo autor.
- Cobertura de tests automatizados de backend (`AlojamientoService`, `UsuarioService`, `EstadisticasController`) — decisión explícita de no tenerlos todavía.

Seguimiento

A medida que se cierra cada subtarea, se transiciona en Jira (SCRUM, proyecto "Yo, Eloy") de "Por hacer" a "Finalizado". El progreso de este archivo debe reflejar siempre el estado real del repositorio, no un plan aspiracional — si algo cambia de alcance al implementarlo, se actualiza acá.